Movies Data Manager Using Data Structures

Dawar Ahmed
Roll No: i242533
Department of Artificial Intelligence and Data Science
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i242533@nu.edu.pk

Aarab Malik
Roll No: i242552
Department of Artificial Intelligence and Data Science
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i242552@nu.edu.pk

***Abstract*—This report presents our term project which is a Movies Data Manager. We designed this system to manage a large dataset of movies from IMDb. The main goal was to implement core data structures manually without using the Standard Template Library. We used an AVL Tree to store movies and keep them sorted. We used Hash Tables to store actors and genres for fast searching. We also implemented a Graph where movies are connected if they share actors. This helps us provide movie recommendations. The system is written in C++ and handles data parsing searching and pathfinding efficiently.**

***Index Terms*—AVL Tree, Hash Table, Graph, BFS, DFS, C++, Recommendation System.**

I. Introduction

The purpose of this project is to apply data structures to a real world problem. We were tasked with managing a large dataset of movies. The system needs to perform operations like searching for a movie title or finding all movies by a specific actor. To ensure we met the correct theoretical requirements we followed the guidelines from our lecture slides by Ms. Umarah [1].

We also needed to find connections between movies. For example we can check if two movies share the same actors. To achieve this we implemented several custom classes. We did not use any built in containers like vectors or maps. Every node and list was created from scratch. This helped us understand how memory management works in C++.

II. System Architecture

The system is built around several key classes. Each class manages a specific part of the data.

A. Movie and Actor Nodes

We created a `movieNode` class to hold the data. It stores the title and director and rating and release year. It also has pointers to linked lists for genres and actors. We used a `genreNode` struct to make a linked list of genres. We also created an `ActorNode` class. This class stores the actor name and a list of movies they have appeared in. This allows us to traverse from an actor back to their movies easily.

B. Hash Tables for Fast Access

Searching through a list one by one is slow. To fix this we implemented two hash tables. One is the `ActorHashTable` and the other is the `GenreHashTable`. The hash function takes the ASCII sum of the characters in the key and finds the index using the modulo operator. We used chaining to handle collisions. This means if two keys map to the same index we store them in a linked list at that index.

C. AVL Tree for Storage

We used an AVL Tree to store the main collection of movies. An AVL tree is a self balancing binary search tree. This ensures that the height of the tree remains small. When we insert a movie we compare its title to find the correct spot. If the tree becomes unbalanced we perform rotations. We consulted GeeksforGeeks to verify our logic for these rotations [2]. We implemented both left and right rotations to keep the balance factor stable. This makes searching for a movie by title very fast.

D. Graph Implementation

The most complex part of the system is the Graph. The graph helps us find relationships between movies. In our `Graph` class every movie is treated as a vertex. We create an edge between two movies if they have something in common. We check if they share any actors to add a connection. We used an adjacency list to represent the graph. This is an array of linked lists where each index represents a movie and the list contains its neighbors.

III. Methodology

Our approach involves several steps to process the data efficiently.

A. Data Parsing and Cleaning

We read the data from a CSV file. We look at each line character by character. A major challenge was dealing with messy data. We implemented a `cleanString` function to handle this. It filters out non printable characters and skips UTF8 continuation bytes to avoid encoding errors. Algorithm 1 shows how we prepare the strings before storage.

B. Recommendation Logic

For recommendations we use graph traversals. If a user asks for recommendations based on a movie we look at the graph neighbors. BFS is good for finding similar movies that are closely related. DFS is used when we want to explore a specific path deeply. We learned the concepts for these traversals from

Abdul Bari's videos [3]. We limit the depth of the search to avoid taking too long.

IV. Algorithms

We implemented specific algorithms for data processing and graph traversal.

Algorithm 1: String Cleaning and Trimming

Input: Raw String S
Output: Cleaned String
$cleaned \leftarrow$ ""
for *each character* c *in* S **do**
  if *c is between ASCII 32 and 126* **then**
    Append c to $cleaned$
  end
end
$start \leftarrow 0$, $end \leftarrow length(cleaned) - 1$
while *cleaned[start] is whitespace* **do**
  $start \leftarrow start + 1$
end
while *cleaned[end] is whitespace* **do**
  $end \leftarrow end - 1$
end
return Substring of $cleaned$ from $start$ to end

A. Graph Construction

Building the graph is a crucial step. We connect movies if they share common attributes. Algorithm 2 details this process.

Algorithm 2: Building the Movie Graph

Input: AVL Tree T
Output: Adjacency List adj
$count \leftarrow T.getMovieCount()$
Collect all movies from T into array M
for $i \leftarrow 0$ **to** $count - 1$ **do**
  for $j \leftarrow i + 1$ **to** $count - 1$ **do**
    if *shareActors(*$M[i]$, $M[j]$*)* **then**
      $addEdge(i, j)$
      $addEdge(j, i)$
    end
  end
end

B. Recommendation Engine

We used BFS to recommend movies. This finds movies that are directly connected (distance 1) or connected via another movie (distance 2). Algorithm 3 shows this logic.

V. Discussion

We faced some challenges during the project. One issue was memory management. Since we used `new` to create nodes we had to ensure we used `delete` to free the memory. We wrote destructors for every class. The destructors traverse the linked lists and delete every node one by one.

Another challenge was duplicate data. Sometimes the same actor appears multiple times. We used the `getOrCreate` function in the hash table. This function checks if the actor already exists. If they do it returns the existing node. If not it creates a new one. This keeps our data unique and clean.

Algorithm 3: BFS Recommendations

Input: Start Node src, Max Depth D
Output: Printed Recommendations
Create Queue Q
$visited[src] \leftarrow true$
$distance[src] \leftarrow 0$
$Q.enqueue(src)$
while Q *is not empty* **do**
  $current \leftarrow Q.dequeue()$
  if *distance[current]* $\geq D$ **then**
    Continue
  end
  $temp \leftarrow adj[current]$
  while $temp$ *is not null* **do**
    $neighbor \leftarrow temp.vertexIndex$
    if *visited[neighbor] is false* **then**
      $visited[neighbor] \leftarrow true$
      $distance[neighbor] \leftarrow$
       $distance[current] + 1$
      $Q.enqueue(neighbor)$
    end
    $temp \leftarrow temp.next$
  end
end
Print movies ordered by distance

VI. Conclusion

In conclusion we successfully built a Movies Data Manager. The system uses an AVL tree for efficient storage and sorting. It uses hash tables for quick lookups of actors and genres. The graph structure allows us to find interesting connections between movies. We verified that all the requirements were met without using STL. This project helped us understand the internal working of data structures better.

Acknowledgment

We would like to thank our instructors at the National University of Computer and Emerging Sciences for their guidance.

References

[1] M. Umarah, "Data structures lecture slides," National University of Computer and Emerging Sciences, 2025, course Material.
[2] "Geeksforgeeks data structures," https://www.geeksforgeeks.org/, 2025, accessed for AVL Tree logic.
[3] A. Bari, "Algorithms and data structures," YouTube Channel, 2025, accessed for Graph Traversal concepts.